

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*
(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I **BALAJI D** (Name) / **192512368** (Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Department: **B E ECE**

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **BALAJI D**

Q1. Hospital Shift Assignment – Backtracking Search (CSP)

Problem Formulation

- Variables: D1, D2, D3 (the three doctors).
- Domain of each variable: {Morning, Afternoon, Night}.
- Constraints:
 - (i) $D1 \neq \text{Night}$
 - (ii) $\text{Shift}(D2) < \text{Shift}(D3)$, where Morning < Afternoon < Night
 - (iii) All-Different(D1, D2, D3) – only one doctor per shift
 - (iv) $D3 \neq \text{Morning}$

Variable Ordering (Most-Constrained-Variable Heuristic)

D3 is involved in the greatest number of constraints (constraints ii and iv both restrict D3), so D3 is assigned first, followed by D1, and finally D2. This ordering keeps the search tree small and demonstrates backtracking clearly.

Backtracking Search – Step-by-Step Trace

1. Assign D3. Domain after applying constraint (iv), $D3 \neq \text{Morning}$: {Afternoon, Night}.
2. Try D3 = Afternoon. (Consistent so far – satisfies constraint iv.)
3. Assign D1. Domain after removing Afternoon (taken) and applying $D1 \neq \text{Night}$: {Morning}.
4. Try D1 = Morning. (Consistent – satisfies constraint i.)
5. Assign D2. Only shift left is Night, so try D2 = Night.
6. Constraint check (ii): $\text{Shift}(D2) < \text{Shift}(D3) \rightarrow \text{Night} < \text{Afternoon}$ is FALSE. Constraint violated – FAILURE.
7. Backtrack: undo D2 = Night. No other value left for D2 $\rightarrow$ backtrack further.
8. Backtrack: undo D1 = Morning. No other value left for D1 (domain exhausted) $\rightarrow$ backtrack further.
9. Backtrack: undo D3 = Afternoon. Try next value in domain: D3 = Night. (Consistent – satisfies constraint iv.)
10. Assign D1. Domain after removing Night (taken) and applying $D1 \neq \text{Night}$ (already satisfied): {Morning, Afternoon}. Try D1 = Morning. (Consistent.)
11. Assign D2. Only shift left is Afternoon, so D2 = Afternoon.
12. Constraint check (ii): $\text{Shift}(D2) < \text{Shift}(D3) \rightarrow \text{Afternoon} < \text{Night}$ is TRUE. All constraints (i)–(iv) satisfied – GOAL REACHED.

Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Verification: D1 (Morning) $\neq$ Night – satisfied. D2 (Afternoon) is before D3 (Night) – satisfied. Morning, Afternoon and Night are assigned to exactly one doctor each – satisfied. D3 (Night) $\neq$ Morning – satisfied. All doctors have exactly one shift – satisfied.

Q2. Robot Navigation in a 5×5 Grid – Search with Manhattan Heuristic

Problem Formulation

- State: (row, column) position of the robot on the 5×5 grid, rows and columns numbered 0 to 4.
- Initial State S = (0, 0). Goal State G = (4, 4).
- Actions: Up, Down, Left, Right – each with a step cost of 1.
- Obstacles (impassable cells): (0,2), (1,2), (2,2), (3,2).
- Heuristic: $h(n) = \text{Manhattan distance from } n \text{ to } G = |4 - \text{row}| + |4 - \text{col}|$.

Grid Layout

	C0	C1	C2	C3	C4
R0	[S]	[.]	[X]	[.]	[.]
R1	[.]	[.]	[X]	[.]	[.]
R2	[.]	[.]	[X]	[.]	[.]
R3	[.]	[.]	[X]	[.]	[.]
R4	[.]	[.]	[.]	[.]	[G]

S = Start (0,0) G = Goal (4,4) X = Obstacle

Search Method Used

Since the problem specifies a Manhattan-distance heuristic together with a priority queue, the appropriate BFS-based algorithm here is A* Search – an extension of breadth-first / uniform-cost expansion where the priority queue orders nodes by $f(n) = g(n) + h(n)$, $g(n)$ being the path cost so far and $h(n)$ the Manhattan-distance estimate to the goal.

Step-by-Step Node Expansion (Priority Queue Ordered by $f = g + h$)

Iter.	Node (r,c)	$g(n)$	$h(n)$	$f = g+h$	Remark
1	(0,0)	0	8	8	Start node expanded
2	(1,0)	1	7	8	Down from (0,0)
2	(0,1)	1	7	8	Right from (0,0)
3	(2,0)	2	6	8	Down from (1,0)
3	(1,1)	2	6	8	Right from (1,0)
4	(3,0)	3	5	8	Down from (2,0)
4	(2,1)	3	5	8	Right from (2,0)/(1,1)
5	(4,0)	4	4	8	Down from (3,0)
5	(3,1)	4	4	8	Right from (3,0)/(2,1)
6	(4,1)	5	3	8	Right from (4,0)/(3,1)
7	(4,2)	6	2	8	Right from (4,1) – gap in obstacle wall
8	(4,3)	7	1	8	Right from (4,2)
9	(4,4)	8	0	8	GOAL reached

Note: at every layer several nodes tie on $f = 8$ because the Manhattan-distance heuristic is exact for this open grid (there is no obstacle that forces the robot to move away from the goal). This makes A* expand nodes in increasing

order of $g(n)$ – i.e. layer by layer, exactly like breadth-first search – while the heuristic still correctly guides the tie-breaking towards the goal side of the grid.

Optimal Path and Cost

Path: $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$

Moves: Down, Down, Down, Down, Right, Right, Right, Right

Total path cost = 8 (8 moves $\times$ cost 1 per move), which equals the initial heuristic estimate $h(S) = 8$, confirming the path found is optimal.

Q3. Autonomous Rescue Robot – Search in a Partially Observable Environment

(a) Analysis of Constraints and Their Effect on Decision-Making

- Movement restricted to Up/Down/Left/Right: reduces the branching factor at every node to at most 4, simplifying the search but also limiting the paths available around obstacles.
- Blocked cells / obstacles: the robot cannot know the full obstacle layout in advance, so it must treat unexplored cells as “unknown” rather than “free”, forcing cautious, incremental decisions.
- Limited sensing (only adjacent cells observable): the environment is partially observable, so the agent cannot plan a complete optimal path offline – it must interleave sensing, planning and acting.
- Cost model (1 for a normal move, 2 for entering a risky zone): this turns the problem into a weighted least-cost search rather than a simple shortest-path/BFS problem, since the number of moves alone no longer determines the best path.
- Requirement to minimise total cost while ensuring safe navigation: creates a trade-off the agent must resolve at every step – a shorter path through a risky zone may cost more than a longer path through normal cells.
- Dynamic/online knowledge update: since the map is revealed only as the robot moves, previously assumed “best” paths may need to be revised the moment a new obstacle or risky zone is sensed.

(b) Solution Approach – Online Search Agent (Online Uniform-Cost / LRTA*-style Search)

Because the environment is only partially observable, a standard offline Uniform Cost Search (which needs the complete graph and all edge costs known in advance) cannot be applied directly. Instead, an Online Search Agent is used, which interleaves computation with action execution:

1. Initialise the agent at S with cost-so-far $g = 0$ and an empty internal map (nothing known beyond the starting cell).
2. Sense the environment: observe the status (free / blocked / risky) of all currently adjacent cells, and update the internal map with this new information.
3. Evaluate each reachable neighbour using an estimated total cost $f = g(n) + \text{step-cost}(n) + h(n)$, where step-cost is 1 for a normal cell and 2 for a risky cell, and $h(n)$ is a Manhattan-distance estimate to G.
4. Move to the neighbour with the lowest estimated f value (a locally optimal, uniform-cost style choice), and update g accordingly.
5. If, after moving, sensing reveals that the chosen direction is a dead end or newly discovered obstacle, the agent backtracks to the last cell with an unexplored alternative and updates its internal map so that route is not tried again.
6. Repeat the sense $\rightarrow$ evaluate $\rightarrow$ move $\rightarrow$ update cycle until the goal G is reached, accumulating the true total path cost actually incurred (not merely estimated).
7. Report the final path taken and the total cost, which reflects the real costs of the cells traversed, including any risky-zone penalties.

(c) AI Concepts Demonstrated by the Robot

- Intelligent Agent: the rescue robot is a goal-based / utility-based agent – it perceives its immediate surroundings through limited sensors, maintains an internal (continuously updated) model of the explored building, and chooses actuator actions (moves) that are expected to minimise total cost while making progress toward the goal G.
- Rationality: the robot is rational because, given the percepts and knowledge available to it at each step, it always selects the action expected to be best (lowest expected total cost while remaining safe) – it is not required to be globally optimal from the start, since perfect knowledge of the whole building is unavailable; this is bounded/limited rationality under partial observability.

- Environment Type: Partially Observable (only adjacent cells sensed), Dynamic in terms of the agent's own knowledge (the known map grows as it explores), Sequential (each move affects future options), Discrete (grid-based states and actions), and Single-Agent.

(d) Justification of the Chosen Approach

An online search / online uniform-cost (LRTA*-style) agent is the most suitable approach for this problem because the building's obstacle layout and risky zones are not fully known in advance – the robot can only sense adjacent cells. A pure offline Uniform Cost Search would require the complete weighted graph before any move is made, which contradicts the partially observable, limited-sensing constraint given in the problem. By interleaving sensing, cost evaluation and movement, the online approach lets the robot safely discover the building while still using the same least-cost principle as UCS at each local decision, correctly accounting for the higher cost (2) of risky zones and adapting immediately whenever new information contradicts its previous plan – which is exactly the dynamically-updating, online-search-scenario behaviour the question specifies.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided